

# Algorytmy Metaheurystyczne

Algorytm Wyżarzania i Tabu Search

Mateusz Smuga

25.04.2026

## Metody

### Dane

Do wykonania zadania użyto danych ze [strony wydziału matematyki Uniwersytetu Waterloo](#). Wszystkie użyte pliki znajdują się w folderze `assets/`. W ramach pracy wykorzystano zbiory danych:

| Zbiór Danych   | Liczba Miast | Nazwa Pliku |
|----------------|--------------|-------------|
| Western Sahara | 29           | wi29.tsp    |
| Djibouti       | 38           | dj38.tsp    |
| Quatar         | 194          | qa194.tsp   |
| Uruguay        | 734          | uy734.tsp   |
| Zimbabwe       | 929          | zi929.tsp   |
| Oman           | 1979         | mu1979.tsp  |
| Canada         | 4663         | ca4663.tsp  |
| Tanzania       | 6117         | tz6117.tsp  |
| Egypt          | 7146         | eg7146.tsp  |
| Ireland        | 8246         | ei8246.tsp  |

### Otoczenie

Do obu metod dobrano otoczenie 2-optymalne, gdzie rozcinamy dwie krzyżujące się krawędzie i je rozplątujemy licząc na krótszą całkowitą trasę. Zostało ono zaimplementowane w pliku `math_util.rs` wraz z potrzebnymi funkcjami.

### Metoda Symulowanego Wyżarzania

Metoda wykorzystuje pojęcia temperatury, epoki oraz otoczenia. Wszystkie omawiane fragmenty kodu w tej sekcji znajdują się w plikach `annealing.rs` i `simulate_annealing.rs`.

#### Temperatura

Temperatura jest funkcją „czasu”, która pozwala nam wykonywać nie opłacalne posunięcia, aby nie zatrzymywać się w maksimum lokalnym. Z upływem czasu spada i „złe” ruchy zdarzają się coraz rzadziej. W tej implementacji temperatura początkowa jest dobierana na podstawie rozmiaru danych i pewnej stałej dobieranej przez użytkownika.

```
impl SimulationConstants {  
    pub fn from(p: &SimulationParameters) -> Self {  
        let n = p.size as f64;  
  
        let base_temperature = p.base_temperature_k * n.ln();  
    }  
}
```

base\_temperature\_k – parametr dobierany przed uruchomieniem algorytmu

n – rozmiar wczytanych danych

Sama temperatura jest odpowiednio aktualizowana przy użyciu funkcji update\_temperature :

```
pub fn update_temperature(  
    temperature: &mut f64,  
    sim_consts: &SimulationConstants,  
) {  
    *temperature *= sim_consts.temp_constant.exp();  
}
```

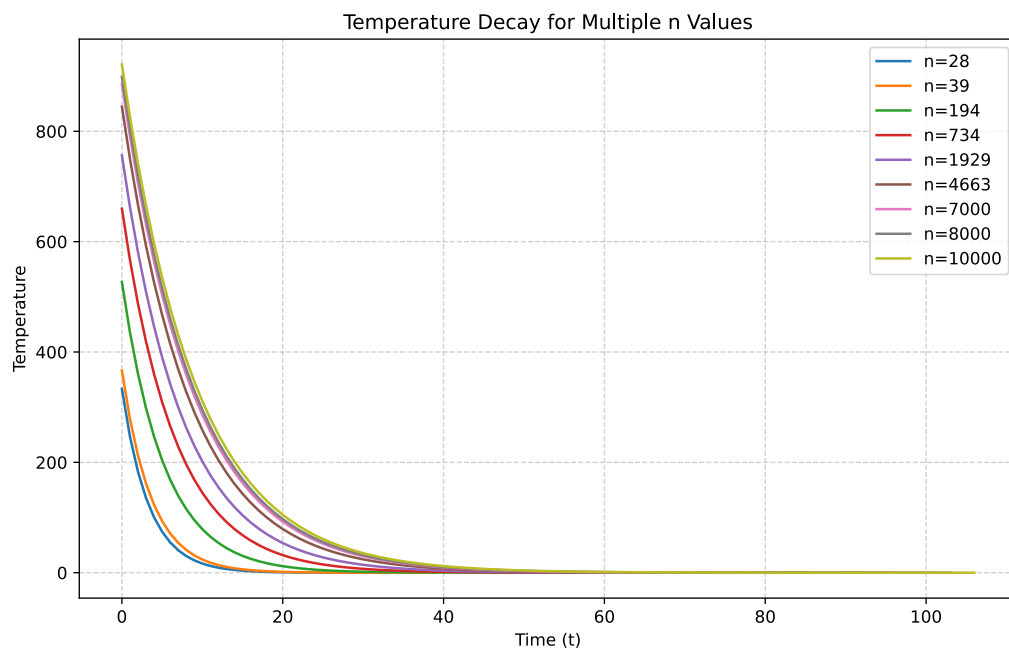
Uwaga:

```
let tau = p.tau_k * base_temperature;  
let temp_constant = (-1.0)/n.ln();  
let min_delta = p.min_delta;
```

Tę rekurencyjną zależność można opisać wzorem:

$$T(t, n) = \begin{cases} k \cdot \ln(n) & : t = 0 \\ T(t-1, n) \cdot e^{-\frac{1}{\ln(n)}} & : t > 0 \end{cases}$$

Przygotowano też wykres porównujący zachowania funkcji temperatury dla różnych rozmiarów danych i standardowo przyjętych stałych:



## Epoka

Epoką nazywamy wszystkie iteracje z tą samą temperaturą. Tutaj liczba iteracji w danej epoce można dostosować następującymi parametrami:

```
pub epoch_min: u64,
pub epoch_max: u64,
pub epoch_range: u64,
```

Są one dobierane w następujący sposób:

```
let epoch_min = (p.epoch_k * n) as u64;
let epoch_max = (5.0 * p.epoch_k * n) as u64;
let epoch_range = epoch_max - epoch_min;
```

epoch\_k – parametr dobierany przez użytkownika

### Warunek Zatrzymania

Warunek zatrzymania jest dość trywialny w porównaniu z doбором funkcji temperatury, mianowicie wystarczy, że temperatura spadnie wystarczająco nisko lub przez ostatnie 50 epok nie zauważymy poprawy:

```
pub fn should_stop(
    temp: f64,
    avg_delta: f64,
    no_improve_epochs: u64,
    p: &SimulationConstants,
) -> bool {
    temp < 0.001 || no_improve_epochs >= 50
}
```

### Działanie Algorytmu

Działanie algorytmu można opisać następującymi krokami:

1. Wylosuj rozwiązanie początkowe  $X$
2. Wybierz losowe rozwiązanie  $X'$  znajdujące się w sąsiedztwie  $X$ .
3. Jeśli  $X'$  jest lepsze to je przyjmij. W przeciwnym razie wyznacz prawdopodobieństwo

przyjęcia  $X'$  używając wzoru:

$$\exp\left(\frac{f(X) - f(X')}{T}\right)$$

i z dokładnością tym prawdopodobieństwem  $X := X'$ .

4. Jeśli nie skończyła się epoka wróć do punktu drugiego.
5. Zmniejsz temperaturę.
6. Jeśli nie osiągnięto warunku stopu wróć do punktu drugiego.

W kodzie wygląda to w taki sposób:

```
pub fn run_simulation(cities: &[City], output_path: &str) -> f64 {
    let n = cities.len();
    if n == 0 { return 0.0; }

    let params = SimulationParameters::new(n as u64);
    let constants = SimulationConstants::from(&params);
    let dist_matrix = create_distance_matrix(cities);

    let mut current_path = generate_random_path(n);
    let mut current_distance = calculate_total_distance(&current_path, &dist_matrix);

    let mut best_path = current_path.clone();
    let mut best_distance = current_distance;
```

```

    let mut temperature = constants.base_temperature;
    let mut no_improve_epochs = 0;
    let mut total_iterations = 0;

    println!("Starting SA for {} cities. Initial distance: {:.2}", n,
current_distance);

    while !should_stop(temperature, current_distance, no_improve_epochs, &constants)
    {
        let mut improved_in_epoch = false;
        let current_epoch_len = epoch_length(temperature, &constants);

        for _ in 0..current_epoch_len {
            let (i, j) = get_random_indices(n);

            let delta = calculate_2opt_delta(&current_path, i, j, &dist_matrix);

            if accept(delta, temperature, &constants) {
                apply_2opt(&mut current_path, i, j);
                current_distance += delta;

                if current_distance < best_distance {
                    best_distance = current_distance;
                    best_path = current_path.clone();
                    improved_in_epoch = true;
                }
            }
            total_iterations += 1;
        }

        if improved_in_epoch {
            no_improve_epochs = 0;
        } else {
            no_improve_epochs += 1;
        }

        update_temperature(&mut temperature, &constants);
    }

    println!("Simulation finished. Best distance: {:.2} (Total iterations: {})",
best_distance, total_iterations);

    if let Err(e) = save_tour(&best_path, output_path) {
        eprintln!("Failed to save tour: {}", e);
    }

    best_distance
}

```

## Dobór Parametrów

Parametry wykorzystywane w metodzie symulowanego wyżarzania zostały dobrane empirycznie przy ręcznym testowaniu, tak aby wyniki zbiegały do optymalnych rozwiązań w jak najmniejszym czasie. Dobrane stałe:

```
impl SimulationParameters {
    pub const fn new(size: u64) -> Self {
        Self {
            size,
            base_temperature_k: 100.0,
            epoch_k: 500.0,
            tau_k: 1.0,
            epsilon: 1e-7,
            min_delta: -1e-9,
        }
    }
}
```

## Metoda Tabu Search

### Algorytm

Algorytm rozwija ideę Local Search poprzez dodanie listy tabu i sprytniejszego przeszukiwania otoczenia. Działanie algorytmu można opisać następująco:

1. Losujemy początkowe rozwiązanie
2. Przeszukujemy jego otoczenie i wybieramy najlepsze rozwiązanie, którego nie ma na liście tabu
3. Umieszczamy je na liście i przyjmujemy jako bieżące rozwiązanie
4. Sprawdzamy warunek stopu

W kodzie odpowiada temu metoda z pliku `tabu_search.rs`.

### Warunek Zatrzymania

Warunkiem zatrzymania jest wykonanie maksymalnej liczby iteracji.

### Sposób Wyboru Sąsiada

Sąsiada do analizy wybieramy w sposób losowy dobierając dwa różne wierzchołki i próbując wywołać invert.

### Dobór Parametrów

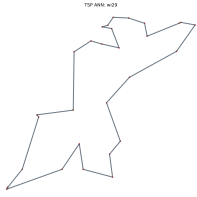
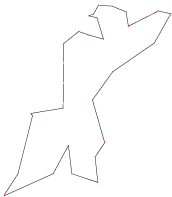
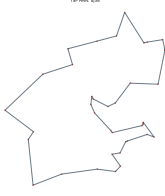
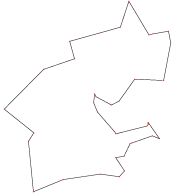
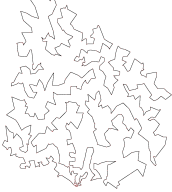
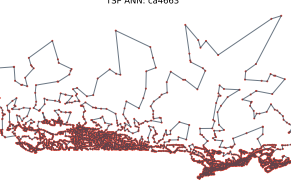
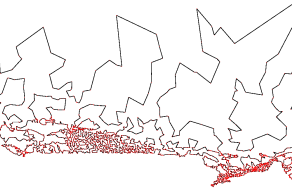
Parametry dobrane empirycznie przy ręcznym testowaniu:

```
let tabu_params = TabuParameters {
    max_iterations: 10000,
    tabu_tenure: (n as f64 * 0.2) as usize,
    neighborhood_size: (n * 2).min(500),
};
```

## Uzyskane Wyniki

### Wyniki Algorytmu Wyżarzania (Simulated Annealing)

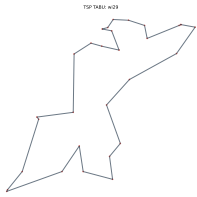
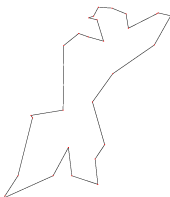
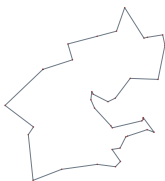
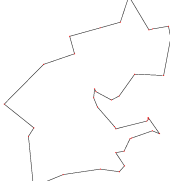
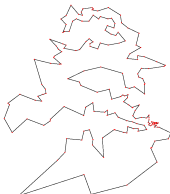
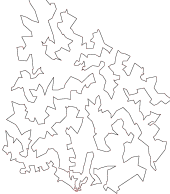
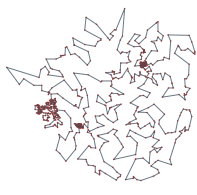
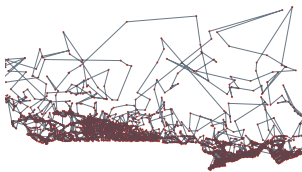
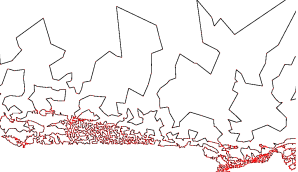
| Instancja (N) | Wyniki (ANN) | Wynik Optymalny | Długość znaleziona | Długość optymalna |
|---------------|--------------|-----------------|--------------------|-------------------|
|---------------|--------------|-----------------|--------------------|-------------------|

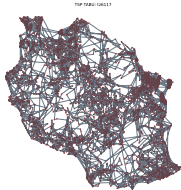
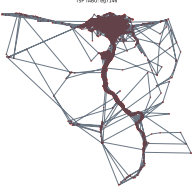
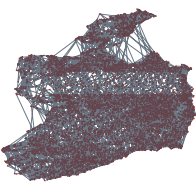
|                                 |                                                                                     |                                                                                     |            |         |
|---------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|------------|---------|
| <b>Western Sahara</b><br>(wi29) |    |    | 27601.17   | 27603   |
| <b>Djibouti</b><br>(dj38)       |    |    | 6659.43    | 6656    |
| <b>Qatar</b><br>(qa194)         |    |    | 9437.73    | 9352    |
| <b>Uruguay</b><br>(uy734)       |   |   | 81324.27   | 79114   |
| <b>Zimbabwe</b><br>(zi929)      |  |  | 97443.77   | 95345   |
| <b>Oman</b><br>(mu1979)         |  |  | 89769.91   | 86891   |
| <b>Canada</b><br>(ca4663)       |  |  | 1355651.52 | 1290319 |

|                             |                                                                                   |                                                                                   |           |        |
|-----------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------|--------|
| <b>Tanzania</b><br>(tz6117) |  |  | 417762.54 | 259045 |
| <b>Egypt</b><br>(eg7146)    |  |  | 181049.5  | 501507 |
| <b>Irlandia</b><br>(ei8246) |  |  | 219994.85 | 206171 |



## Wyniki Algorytmu Tabu Search

| Instancja (N)                   | Wyniki (TABU)                                                                                        | Wynik Optymalny                                                                     | Długość znaleziona | Długość optymalna |
|---------------------------------|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------|-------------------|
| <b>Western Sahara</b><br>(wi29) |  TSP TABU: w29      |    | 27601.17           | 27603             |
| <b>Djibouti</b><br>(dj38)       |  TSP TABU: dj38     |    | 6659.43            | 6656              |
| <b>Qatar</b><br>(qa194)         |  TSP TABU: qa194   |   | 10039.7            | 9352              |
| <b>Uruguay</b><br>(uy734)       |  TSP TABU: uy734  |  | 93830.3            | 79114             |
| <b>Zimbabwe</b><br>(zi929)      |  TSP TABU: zi929  |  | 109839.34          | 95345             |
| <b>Oman</b><br>(mu1979)         |  TSP TABU: mu1979 |  | 134567.19          | 86891             |
| <b>Canada</b><br>(ca4663)       |  TSP TABU: ca4663 |  | 3980677.91         | 1290319           |

|                             |                                                                                   |                                                                                   |            |        |
|-----------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|------------|--------|
| <b>Tanzania</b><br>(tz6117) |  |  | 1841657.65 | 259045 |
| <b>Egypt</b><br>(eg7146)    |  |  | 965296.17  | 501507 |
| <b>Irlandia</b><br>(ei8246) |  |  | 1479252.09 | 206171 |